

Assignment No.1

SaaS Cloud Using HDFS on Azure

Title of the Experiment:

Setup your own cloud for Software as a Service (SaaS) over the existing LAN in your laboratory. Write your own code for cloud controller using open-source technologies integrated with HDFS. Implement file segmentation (block-wise storage) and perform upload/download of files in encrypted form.

Objectives:

- To build a private cloud infrastructure using Microsoft Azure Virtual Machines connected through a Virtual Network (simulating a LAN environment).
- To deploy a distributed Hadoop HDFS cluster consisting of 1 Master Node and 2 Worker Nodes.
- To implement file storage using HDFS with block-wise distribution and replication.
- To develop a Python Flask-based SaaS Cloud Controller for file upload and download.
- To implement AES-based encryption and decryption for secure data storage.
- To demonstrate secure distributed cloud storage using open-source technologies.

Software & Resources Used:

Cloud Provider: Microsoft Azure (Azure for Students subscription)
Infrastructure: 3 Azure VMs → Master Node (20.235.81.30), Worker Node-1 (20.219.7.28), Worker Node-2 (20.219.189.117)
Virtual Network: VNet (10.0.0.0/16), Subnet (10.0.0.0/24)
Resource Group: SaaS-Assignment-Group
OS: Ubuntu Server 24.04 LTS (All nodes)
VM Size: Standard_B2ats_v2
Distributed Storage: Apache Hadoop 3.3.6 (HDFS: Master + Workers)
Runtime: Java OpenJDK 11
Cloud Controller: Python 3, Flask, cryptography (Fernet AES), hdfs client
Replication Factor: 2 (fault tolerance across workers)
Ports Used: SSH (22), HDFS UI (9870), Flask App (5000), Hadoop RPC (9000)

Theory / Background:

Software as a Service (SaaS) is a cloud model where applications are hosted on the cloud and accessed via the internet, eliminating local installation and maintenance.

Hadoop HDFS (Hadoop Distributed File System) is a distributed storage system that splits large files into blocks and stores them across multiple DataNodes. A NameNode manages metadata and block locations.

In this experiment, replication factor 2 is used, meaning each block is stored on two different DataNodes to ensure fault tolerance. If one node fails, data remains accessible from another node. The fsck tool is used to verify block distribution.

Steps / Procedure:

Phase 1: Azure Cloud Setup

Step 1: Resource Group Creation

A Resource Group named **SaaS-Assignment-Group** was created in Microsoft Azure under the Azure for Students subscription. It acts as a container for all cloud resources such as virtual machines and networking components.

Step 2: Virtual Network Configuration

A Virtual Network (VNet) was created with:

- Address space: **10.0.0.0/16**
- Subnet: **10.0.0.0/24**

This simulates a LAN environment where all VMs can communicate internally.

◆ Phase 2: Virtual Machine Setup

Three Ubuntu VMs were created:

- Master Node (NameNode)
- Worker Node 1 (DataNode)
- Worker Node 2 (DataNode)

All VMs were deployed in the same Virtual Network (10.0.0.0/16) to simulate a LAN environment.

...

☰

Microsoft Azure

Search

Home

Create a virtual machine

...

Help me choose the right VM size for my workload

Help me create

Validation passed

BasicsDisksNetworkingManagementMonitoringAdvancedTagsReview + create

Price

1 X Standard B2ats v2

by Microsoft

Terms of use | Privacy policy

Subscription credits apply

0.0131 USD/hr

Pricing for other VM sizes

TERMS

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. See the [Azure Marketplace Terms](#) for additional details.

Name

PRATHAMESH GHADGE

Preferred e-mail address

S24CE018@ms.pict.edu

Preferred phone number

0

You have set SSH port(s) open to the internet. This is only recommended for testing. If you want to change this setting, go back to Basics tab.

Basics

Subscription

Azure for Students

Resource group

VM-RG

Virtual machine name

vm1

< PreviousNext >Create


Microsoft Azure

Home

Create a virtual machine



 Validation passed

Basics

Subscription	Azure for Students
Resource group	VM-RG
Virtual machine name	vm1
Region	East Asia
Availability options	Availability zone
Zone options	Self-selected zone
Availability zone	3
Security type	Trusted launch virtual machines
Enable secure boot	Yes
Enable vTPM	Yes
Integrity monitoring	No
Image	Ubuntu Server 24.04 LTS - Gen2
VM architecture	x64
Size	Standard B2ats v2 (2 vcpus, 1 GiB memory)
Enable Hibernation	No
Authentication type	SSH public key
Username	azureuser
SSH Key format	RSA
Key pair name	vm1_key
Public inbound ports	SSH
Azure Spot	No

Disks

OS disk size	Image default
OS disk type	Premium SSD LRS
Use managed disks	Yes


Previous


Next


Create

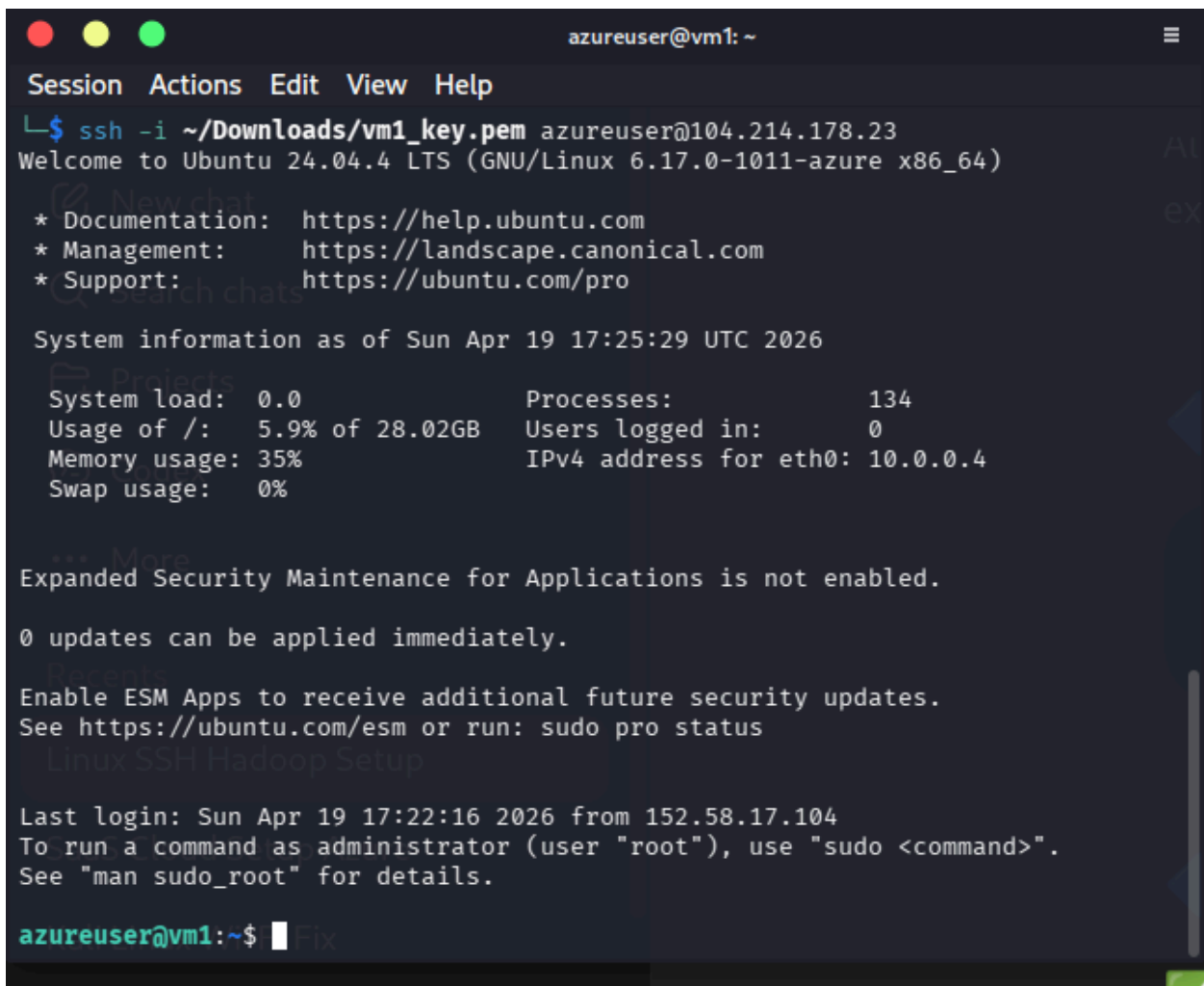
Network Security Configuration (NSG)

Inbound rules were configured on Master VM:

- SSH (22) → Remote access
- HDFS Web UI (9870) → NameNode interface
- Flask App (5000) → SaaS application access
- Hadoop RPC (9000) → Cluster communication

◆ Phase 3: SSH Configuration

Passwordless SSH was configured using RSA keys to enable secure communication between Master and Worker nodes, required for automatic Hadoop service startup.

A terminal window titled 'azureuser@vm1: ~' showing an SSH session. The user has executed 'ssh -i ~/Downloads/vm1_key.pem azureuser@104.214.178.23'. The terminal displays the Ubuntu 24.04.4 LTS welcome message, system information (Sun Apr 19 17:25:29 UTC 2026), and security notices. The system information table shows: System load: 0.0, Processes: 134, Usage of /: 5.9% of 28.02GB, Users logged in: 0, Memory usage: 35%, IPv4 address for eth0: 10.0.0.4, and Swap usage: 0%. The terminal ends with the prompt 'azureuser@vm1:~\$' and a cursor.

```
azureuser@vm1: ~
Session Actions Edit View Help
$ ssh -i ~/Downloads/vm1_key.pem azureuser@104.214.178.23
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1011-azure x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Sun Apr 19 17:25:29 UTC 2026

System load:  0.0              Processes:    134
Usage of /:   5.9% of 28.02GB   Users logged in: 0
Memory usage: 35%              IPv4 address for eth0: 10.0.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun Apr 19 17:22:16 2026 from 152.58.17.104
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@vm1:~$
```

◆ Phase 4: Hadoop Installation

Java OpenJDK 11 and Hadoop 3.3.6 were installed on all nodes. Configuration files (core-site.xml, hdfs-site.xml, workers, hadoop-env.sh) were updated on the Master Node and copied to Workers.

Hadoop Configuration

Inside `hadoop/etc/hadoop/`, the following files were configured:

1. core-site.xml

Defines NameNode address:

fs.defaultFS = hdfs://master:9000

2. hdfs-site.xml

Defines replication factor:

dfs.replication = 2

3. workers file

Lists worker nodes:

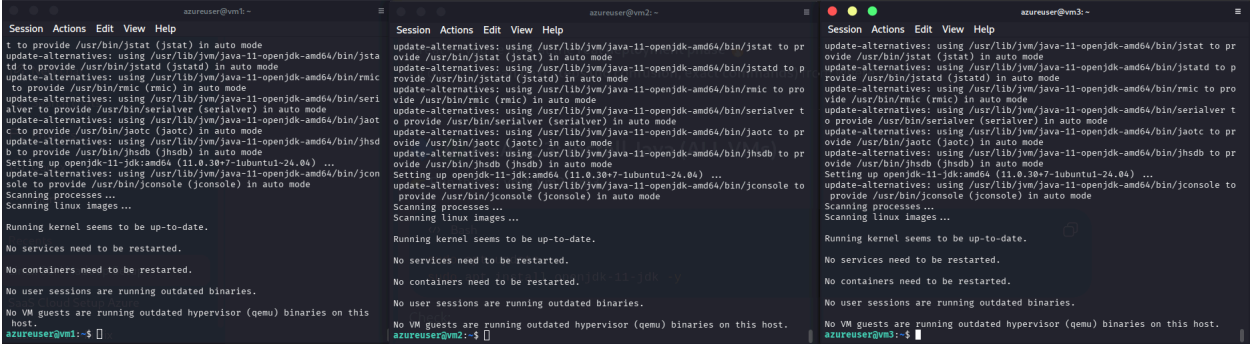
slave1

slave2

4. hadoop-env.sh

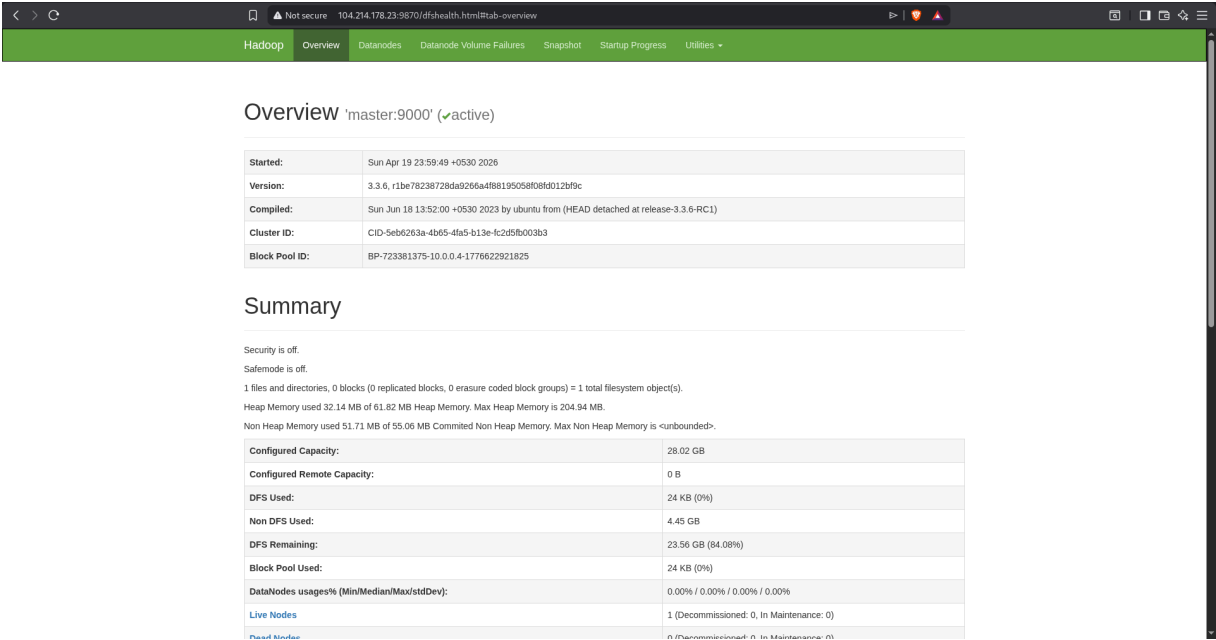
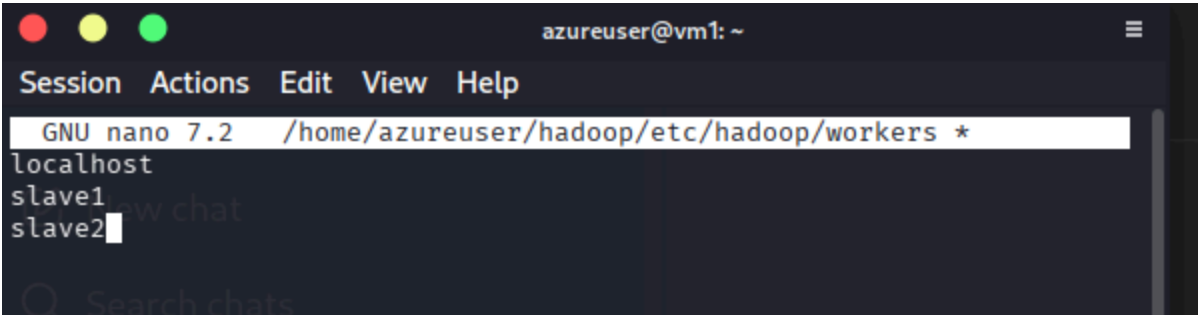
Sets Java path:

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64



Phase 5: HDFS Initialization

The NameNode was formatted using `hdfs namenode -format` and the cluster was started using `start-dfs.sh`. Status was verified using `jps`. The HDFS UI was accessed via port 9870.

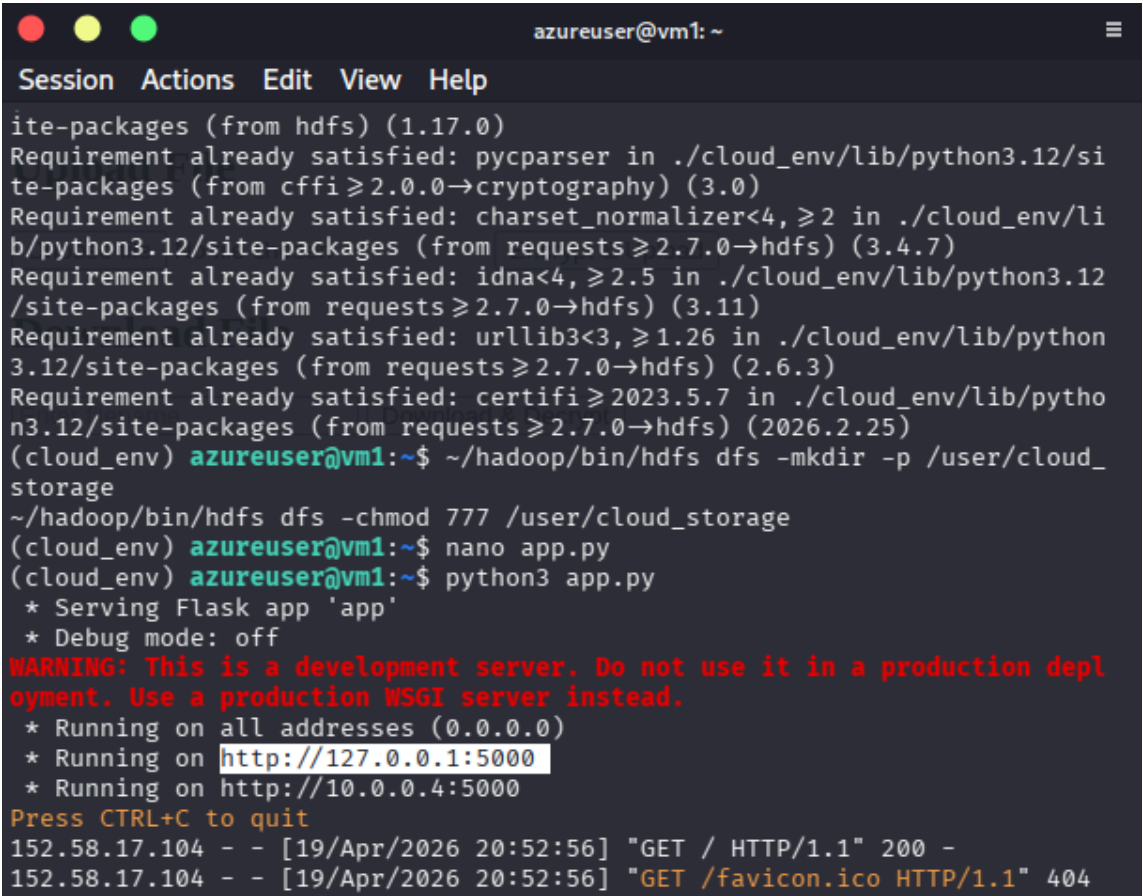


Phase 6: SaaS Cloud Controller

A Flask-based web application was developed on the Master Node. It provides:

- File Upload (Encrypt + Store in HDFS)
- File Download (Retrieve + Decrypt)

The application runs on port 5000 and uses Fernet AES encryption to secure data.



Phase 7: SaaS Interface

The web interface allows users to:

- Upload files securely to cloud storage
- Download decrypted files from HDFS
- Interact with cloud system via browser
-



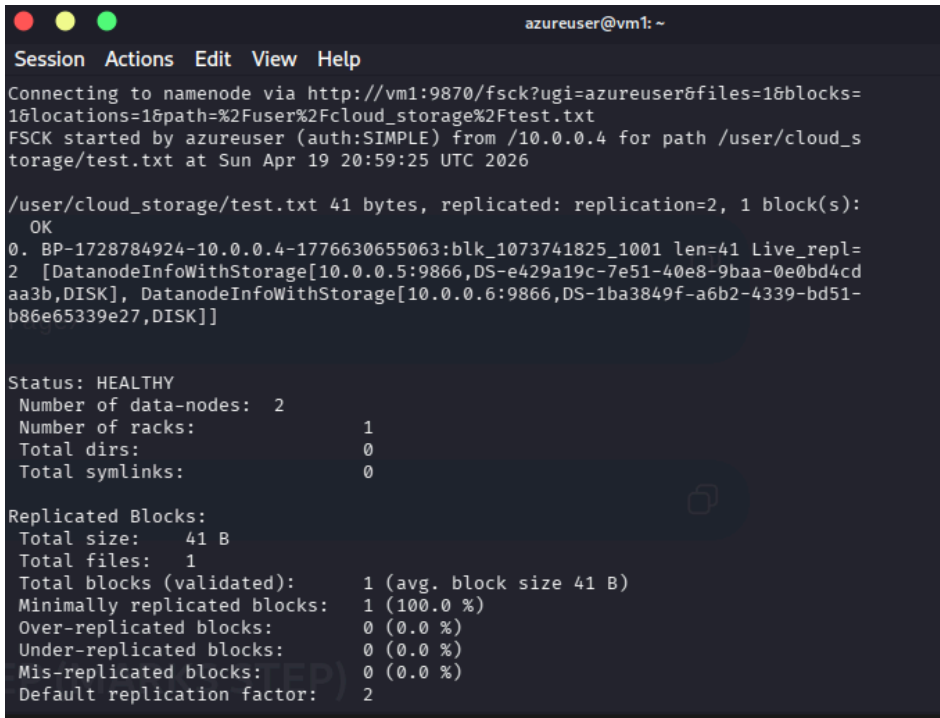
Step 1: File Splitting

Uploaded file is divided into small blocks (manually in Flask or by HDFS).
Example:

file.txt → block-0, block-1, block-2

Step 2: Encryption

Each block is encrypted using AES (Fernet) before storage.
block → encrypted_block



Conclusion:

The experiment successfully demonstrated the implementation of a private SaaS-based cloud storage system using Microsoft Azure, Apache Hadoop HDFS, and Python Flask. A distributed cluster consisting of one Master Node and two Worker Nodes was deployed over a Virtual Network simulating a LAN environment.